

**Universidade Estadual de Santa Cruz — UESC**

Centro de Ciências Exatas e Tecnológicas

Departamento de Ciências Exatas e Tecnológicas

Curso de Ciência da Computação

**Projeto 1 — Benchmark de Suavização/Desfoque Gaussiano  
Iterativo: Implementação Paralela em GPU com CUDA (Etapa  
3)**

*Relatório acadêmico — DEC107 — Processamento Paralelo*

*04 de julho de 2026*

Autores: Lucas Luige Costa Miranda  
Henrique Cerqueira Fadigas

Disciplina: DEC107 — Processamento Paralelo

Ilhéus — BA, 2026

# Sumário

|          |                                                                          |           |
|----------|--------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introdução</b>                                                        | <b>2</b>  |
| <b>2</b> | <b>Metodologia</b>                                                       | <b>2</b>  |
| 2.1      | Implementação Paralela com CUDA . . . . .                                | 2         |
| 2.2      | Ambiente de Execução . . . . .                                           | 4         |
| 2.3      | Procedimentos de Medição de Tempo . . . . .                              | 4         |
| 2.4      | Métricas de Avaliação . . . . .                                          | 4         |
| <b>3</b> | <b>Resultados</b>                                                        | <b>5</b>  |
| 3.1      | Teste de Corretude . . . . .                                             | 5         |
| 3.2      | Tempos de Execução . . . . .                                             | 5         |
| 3.3      | Speedup e Eficiência . . . . .                                           | 7         |
| 3.4      | Escalabilidade . . . . .                                                 | 8         |
| 3.5      | Análise de Ocupância e Configuração de Blocos (opcional/bônus) . . . . . | 9         |
| <b>4</b> | <b>Discussão</b>                                                         | <b>11</b> |
| 4.1      | Corretude . . . . .                                                      | 11        |
| 4.2      | Serial vs CUDA . . . . .                                                 | 11        |
| 4.3      | CUDA vs OpenMP e MPI . . . . .                                           | 11        |
| 4.4      | Gargalos e por que Global venceu Shared . . . . .                        | 11        |
| 4.5      | Impacto da configuração de blocos . . . . .                              | 12        |
| 4.6      | Melhorias futuras . . . . .                                              | 12        |
| <b>5</b> | <b>Conclusão</b>                                                         | <b>12</b> |
| <b>6</b> | <b>Referências</b>                                                       | <b>13</b> |

# 1 Introdução

O objetivo desta etapa é implementar o benchmark de suavização Gaussiana em GPU usando CUDA e comparar o desempenho com a versão serial (*baseline*) e com as versões paralelas anteriores, OpenMP (Etapa 1, memória compartilhada) e MPI (Etapa 2, memória distribuída). A diferença é que OpenMP e MPI usam poucos núcleos de CPU, enquanto a GPU trabalha com paralelismo massivo, ou seja, milhares de *threads* leves rodando o mesmo código sobre pixels diferentes, com a placa funcionando como aceleradora ligada ao *host*.

O filtro Gaussiano é o mesmo das etapas anteriores, uma convolução 2D onde cada pixel vira uma média ponderada dos vizinhos, com pesos tirados de uma distribuição Normal. Com kernel  $3 \times 3$  e  $\sigma = 1,0$  ele é o kernel clássico  $\{1, 2, 1\}/\{2, 4, 2\}/\{1, 2, 1\} \div 16$ . Aplicamos o filtro de forma iterativa, 1000 vezes sobre a mesma imagem, o que deixa as diferenças de desempenho entre as abordagens bem mais visíveis.

Vale relembrar alguns conceitos do modelo CUDA. O *host* é a CPU e sua memória, o *device* é a GPU e sua memória, e os *kernels* são as funções que rodam no *device*, lançadas em uma grade de blocos de *threads*. A GPU tem uma hierarquia de memória própria: a global (grande e lenta), a compartilhada (pequena e rápida, visível só dentro de um bloco), a constante (só leitura, com *cache* de *broadcast*) e os registradores (privados de cada *thread*). Boa parte do trabalho de programar para GPU é escolher o que colocar em cada uma dessas memórias.

A análise de desempenho aqui é importante porque o ganho da GPU é grande, mas não vem de graça. Uma implementação mal feita desperdiça boa parte da placa. Por isso medimos com cuidado, isolando o tempo de *kernel*, repetindo as execuções e comparando sempre contra uma saída de referência.

**Nota sobre o hardware.** Ao contrário do previsto, esta etapa não rodou em servidor remoto, e sim no notebook do próprio grupo. Esse notebook (Intel Core i5-11300H com GTX 1650) é diferente da máquina das Etapas 1 e 2 (AMD Ryzen 7 5700G). Para a comparação entre serial, OpenMP, MPI e CUDA ficar justa, rodamos de novo as quatro implementações neste mesmo notebook. Então os tempos de CPU deste relatório não são os dos relatórios anteriores, foram medidos de novo aqui, e por isso algumas conclusões mudam em relação às etapas passadas, como comentamos na Seção 4.

## 2 Metodologia

### 2.1 Implementação Paralela com CUDA

Implementamos dois kernels para o mesmo filtro, para comparar duas estratégias de uso de memória:

- **blur\_global:** uma *thread* por pixel, lendo os nove vizinhos direto da memória global. É a versão direta, sem otimização manual de memória.
- **blur\_shared:** usa *tiling*. Cada bloco carrega junto um *tile* de  $16 \times 16$  pixels mais a moldura de halo (as *half* linhas e colunas de vizinhos que a convolução precisa) na memória compartilhada, sincroniza com `__syncthreads()` e depois faz a convolução lendo só da memória compartilhada. É o equivalente, na GPU, à troca de halos do MPI.

## Mapeamento de threads (blocos e grade)

O bloco é de  $16 \times 16 = 256$  *threads*, e a grade é dimensionada para cobrir a imagem inteira:

$$\text{grade} = \left( \lceil W/16 \rceil, \lceil H/16 \rceil \right), \quad \text{bloco} = (16, 16).$$

Cada *thread* cuida de um pixel, a partir do índice global  $j = \text{blockIdx.x} \cdot 16 + \text{threadIdx.x}$  e  $i = \text{blockIdx.y} \cdot 16 + \text{threadIdx.y}$ . Escolhemos 256 *threads* por bloco por ser múltiplo de 32 (o tamanho do *warp*), o que evita *warps* incompletos, e porque dá boa ocupância na Turing sem estourar registradores ou memória compartilhada (ver Seção 3.5).

## Uso da hierarquia de memória

- **Global:** guarda as duas cópias da imagem (`d_src` e `d_dst`), trocadas a cada iteração (*double buffering*).
- **Constante:** o kernel Gaussiano ( $3 \times 3$ ) é copiado uma vez só para `__constant__ d_kernel`. Como é só leitura e igual para todas as *threads*, aproveita o *cache* de constante com acesso em *broadcast*.
- **Compartilhada:** no `blur_shared`, o *tile* com halo fica em `__shared__`, o que reduz os acessos repetidos à memória global (cada pixel do interior do *tile* é lido uma vez da global e reaproveitado da memória compartilhada pelos vizinhos).
- **Registradores:** o acumulador da soma e os índices locais de cada *thread*.

## Transferências host ↔ device

Antes do laço de iterações, a imagem é copiada do *host* para o *device* (`cudaMemcpy, HostToDevice`) e o kernel Gaussiano vai para a memória constante (`cudaMemcpyToSymbol`). As 1000 iterações rodam inteiras na GPU, sem tráfego com o *host*. Só no fim, na última repetição, o resultado volta (`cudaMemcpy, DeviceToHost`) para ser salvo em disco. Ou seja, para 1000 iterações há só uma cópia de ida e uma de volta.

## Principais funções CUDA utilizadas

- `cudaMalloc` / `cudaFree`: alocam e liberam os *buffers* de imagem na memória global do *device*.
- `cudaMemcpyToSymbol`: copia o kernel Gaussiano para a memória constante.
- `cudaMemcpy`: transfere a imagem *host*→*device* e o resultado *device*→*host*.
- `cudaGetDeviceProperties`: consulta o nome e as características da GPU para o cabeçalho do *benchmark*.
- `cudaEventCreate` / `cudaEventRecord` / `cudaEventSynchronize` / `cudaEventElapsedTime`: medem o tempo do laço de *kernel* direto no *device* (ver Seção 2.3).
- Lançamento de *kernel* `blur_global<<grade,bloco>>(...)`: dispara a grade de blocos na GPU.

## Tratamento das bordas

Como nas versões serial, OpenMP e MPI, os pixels da borda (as **half** primeiras e últimas linhas e colunas) são copiados sem filtrar, o que mantém a equivalência numérica com a referência. No **blur\_shared**, o carregamento do *tile* preenche com zero as posições que caem fora da imagem (halo implícito). Esses zeros nunca entram em uma saída real, porque as bordas são copiadas e não convoluídas.

## 2.2 Ambiente de Execução

- **GPU:** NVIDIA GeForce GTX 1650 (versão *mobile*, TU117), arquitetura Turing, *compute capability* 7.5 (**sm\_75**), 16 SMs (1024 núcleos CUDA) e 4 GB de memória global GDDR5. Notebook Lenovo IdeaPad Gaming 3i.
- **Host (CPU):** Intel Core i5-11300H @ 3.10 GHz, 4 núcleos físicos e 8 *threads* lógicas (*hyperthreading*), 16 GB de RAM DDR4-3200.
- **Sistema operacional:** Linux em WSL2 (Ubuntu 26.04), com o *driver* NVIDIA repassado do *host* Windows.
- **Driver e Toolkit:** *driver* 591.59 (suporta até CUDA 13.1) e compilador **nvcc** do CUDA Toolkit 13.1.
- **Flags de compilação:** `-O2 -arch=sm_75 -Xcompiler -U_GNU_SOURCE`.

Vale registrar um problema de *toolchain* que atrapalha a reprodutibilidade. O CUDA Toolkit 12.6 não compila neste sistema, porque o **glibc** 2.43 do Ubuntu 26.04 declara as extensões **rsqrt**, **cosp** e **sinp** com uma especificação de exceção incompatível com o **math\_functions.h** do CUDA. A saída foi trocar para o Toolkit 13.1 (compatível com o *driver*) e adicionar a flag `-Xcompiler -U_GNU_SOURCE`, que impede o **glibc** de declarar essas extensões GNU que entram em conflito.

## 2.3 Procedimentos de Medição de Tempo

O tempo é medido com eventos CUDA (**cudaEvent\_t**), cronometrando só o laço das 1000 iterações na GPU. As cópias *host*↔*device* do começo e do fim ficam fora da medição.

Fizemos isso de propósito, para a comparação com as etapas anteriores ficar justa. Nas versões serial, OpenMP e MPI também cronometramos só o cálculo (o laço de iterações), sem contar leitura ou gravação de arquivo. Além disso, como são 1000 iterações sobre os mesmos *buffers* já na GPU, as duas cópias únicas (ida e volta) são desprezíveis perto do tempo total de *kernel*. O tempo que reportamos é, então, o de execução do *kernel*, não o ponta a ponta.

Cada configuração rodou 5 repetições e reportamos a média. Os parâmetros fixos foram 1000 iterações, kernel  $3 \times 3$ ,  $\sigma = 1,0$  e bloco  $16 \times 16$ .

## 2.4 Métricas de Avaliação

- **Speedup:**  $S = T_{\text{serial}}/T_{\text{CUDA}}$ .

- **Eficiência:** a fórmula clássica  $E = S/p$  não encaixa bem em GPU, porque não existe um número de *threads* de CPU para dividir. No lugar, olhamos a ocupância teórica dos SMs (Seção 3.5) e estimamos a eficiência de banda de memória (Seção 3.3).
- **Escalabilidade:** como o tempo e o *speedup* se comportam quando a imagem cresce.
- **Comparação** direta com serial, OpenMP e MPI, todas rodadas neste mesmo notebook.

## 3 Resultados

### 3.1 Teste de Corretude

Para validar, comparamos as imagens de cada versão paralela com a saída do serial, pixel a pixel, usando a imagem  $512 \times 512$  e os mesmos parâmetros. As métricas foram a diferença absoluta máxima, o erro quadrático médio (EQM) e a contagem de pixels diferentes.

Tabela 1: Corretude contra a saída serial (imagem  $512 \times 512$ , 262 144 pixels).

| Implementação | Dif. máx. | EQM      | Pixels diferentes |
|---------------|-----------|----------|-------------------|
| OpenMP        | 0         | 0        | 0                 |
| MPI           | 0         | 0        | 0                 |
| CUDA (global) | 1         | 0,000015 | 4                 |
| CUDA (shared) | 1         | 0,000015 | 4                 |

OpenMP e MPI batem exatamente com o serial, porque usam a mesma aritmética de CPU. As duas versões CUDA diferem em só 4 pixels de 262 144, e por só 1 nível de cinza. Adotamos uma tolerância de 1 nível de cinza, porque a GPU usa *fused multiply-add* (FMA) e soma os termos em uma ordem diferente da CPU, o que muda o arredondamento no último *bit* do `float`. Um EQM de cerca de  $10^{-5}$  com diferença máxima de 1 confirma que está correto, e mostra que não há *bug* de halo no `blur_shared`, já que as duas versões CUDA deram o mesmo resultado.

### 3.2 Tempos de Execução

A Tabela 2 traz os tempos médios das duas versões CUDA contra o serial, e a Figura 1 mostra os *speedups*.

Tabela 2: Tempos médios (s) e *speedup*, 1000 iterações, kernel  $3 \times 3$ ,  $\sigma = 1,0$ , 5 repetições. Tempo CUDA = laço de *kernel* (sem cópias).

| Imagem             | Serial (s) | CUDA global (s) | CUDA shared (s) | Speedup (G / S)             |
|--------------------|------------|-----------------|-----------------|-----------------------------|
| $512 \times 512$   | 0,627      | 0,035           | 0,039           | $18,2\times$ / $16,1\times$ |
| $1024 \times 1024$ | 2,817      | 0,110           | 0,143           | $25,6\times$ / $19,7\times$ |
| $2048 \times 2048$ | 11,448     | 0,438           | 0,581           | $26,1\times$ / $19,7\times$ |
| $4096 \times 4096$ | 44,710     | 1,760           | 2,365           | $25,4\times$ / $18,9\times$ |

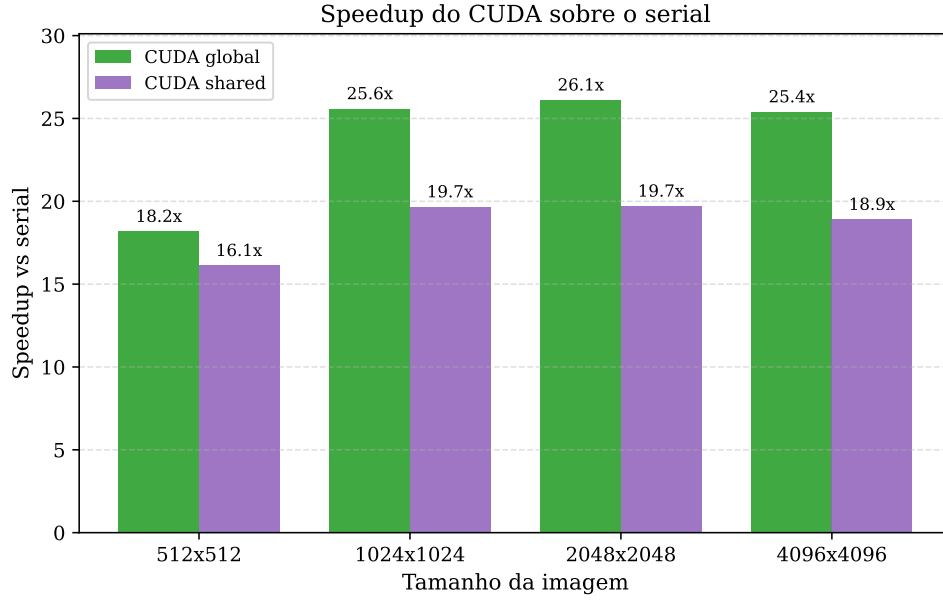


Figura 1: Speedup das versões CUDA (global e shared) sobre o serial.

A Tabela 3 coloca lado a lado a melhor configuração de cada abordagem (na CPU, 8 *threads* ou processos, que é o teto deste hardware de 4 núcleos físicos), e a Figura 2 mostra o comparativo completo na maior imagem, em escala logarítmica.

Tabela 3: Tempos médios (s) das quatro abordagens no mesmo notebook. CPU na melhor configuração (8 *threads* ou processos).

| Imagem      | Serial | OpenMP 8T | MPI 8P | CUDA global | CUDA shared |
|-------------|--------|-----------|--------|-------------|-------------|
| 512 × 512   | 0,627  | 0,286     | 0,552  | 0,035       | 0,039       |
| 1024 × 1024 | 2,817  | 1,050     | 2,196  | 0,110       | 0,143       |
| 2048 × 2048 | 11,448 | 3,450     | 8,079  | 0,438       | 0,581       |
| 4096 × 4096 | 44,710 | 13,017    | 32,741 | 1,760       | 2,365       |

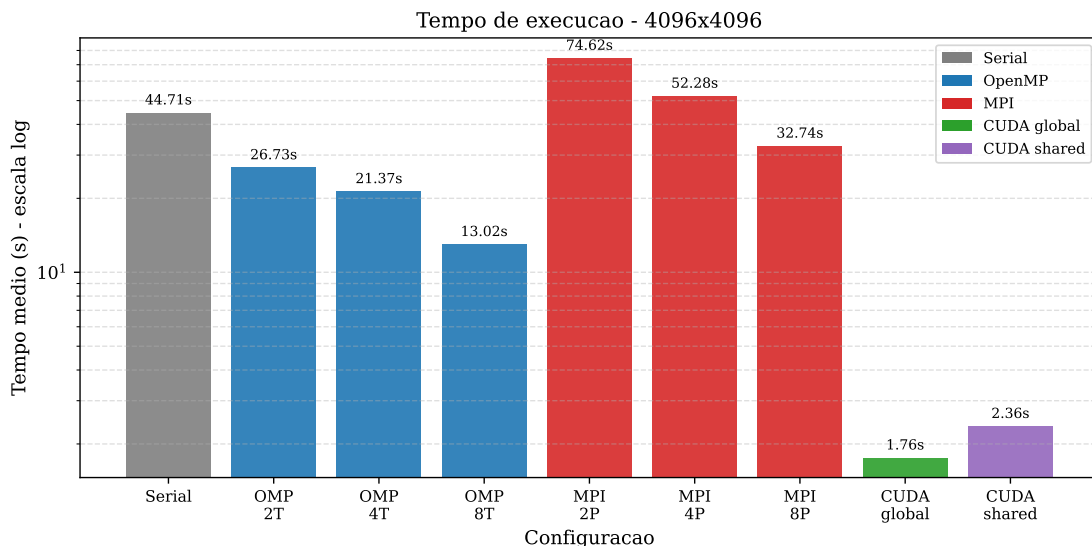


Figura 2: Tempo de execução para a imagem  $4096 \times 4096$  (escala logarítmica).

### 3.3 Speedup e Eficiência

O *speedup* do CUDA sobre o serial (Figura 1) fica entre cerca de  $18\times$  e  $26\times$  na versão global, e entre  $16\times$  e  $20\times$  na *shared*. Em todos os tamanhos de imagem a global foi mais rápida que a *shared* com o kernel  $3 \times 3$  da bateria, um resultado que analisamos na Seção 3.5.

Sobre eficiência, a fórmula clássica  $E = S/p$  não encaixa bem aqui, porque na GPU não há um número de *threads* de CPU para dividir o *speedup*. Mas como o filtro é *memory-bound*, dá para medir uma eficiência que faz sentido, que é quanto da banda de memória da placa o *kernel* usa de fato. Na versão global, cada iteração precisa, no mínimo, ler a imagem uma vez e escrever uma vez na memória global. Para a imagem  $4096 \times 4096$ , com 64 MB por cópia, isso dá 128 MB de tráfego por iteração. Como cada iteração leva 1,76 ms, chegamos a cerca de 76 GB/s. Refazendo a conta no  $2048 \times 2048$  o valor é quase o mesmo, 76 GB/s. O pico teórico da GDDR5 da GTX 1650 é 128 GB/s, então o *kernel* usa pelo menos 60% da banda disponível. Esse valor é um piso, porque o tráfego real é maior que esse mínimo (o *cache* não é perfeito), então na prática a fração é ainda maior. Mesmo sem *profiler*, dá para afirmar que a versão global roda perto do teto de memória da placa, o que combina com o resto da análise.

A Figura 3 mostra essa eficiência de banda para os quatro tamanhos, nas duas versões. A global fica em torno de 60% do  $1024 \times 1024$  pra cima e cai no  $512 \times 512$  (47%), de novo o custo fixo de lançar mil *kernels* sobre um trabalho pequeno. A *shared* fica sempre mais baixa (44% a 46%), coerente com ela ser mais lenta: para o mesmo tráfego de memória, ela leva mais tempo. Reforçando, esses valores são um **piso**, porque assumem que o *cache* L2 absorve o reuso dos vizinhos; o número exato exigiria um *profiler*.



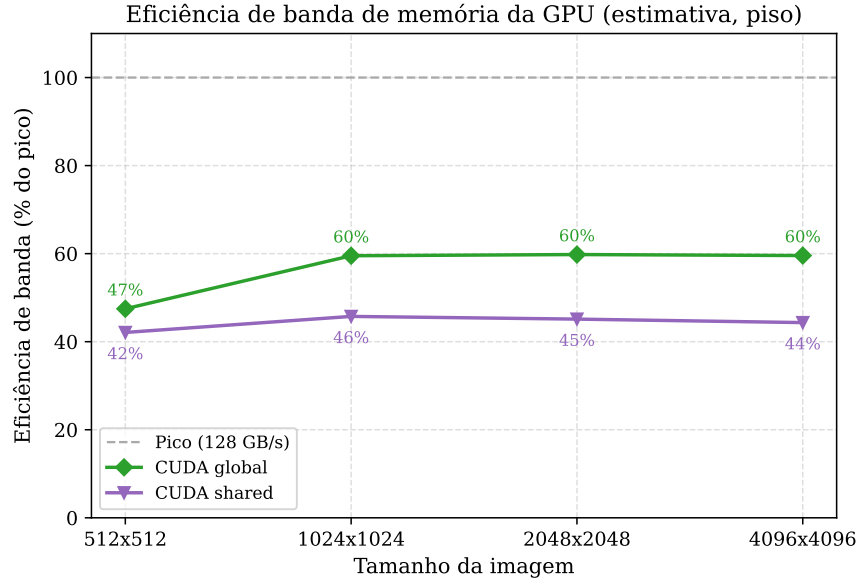


Figura 3: Eficiência de banda de memória da GPU (banda alcançada sobre o pico de 128 GB/s da GDDR5), estimada pelo tráfego mínimo por iteração.

A outra forma de ver o aproveitamento da GPU é a ocupância dos SMs, que discutimos na Seção 3.5.

### 3.4 Escalabilidade

A Figura 4 mostra o tempo das quatro abordagens em função do tamanho da imagem (escala log), e a Figura 5 a escalabilidade em *log-log*.

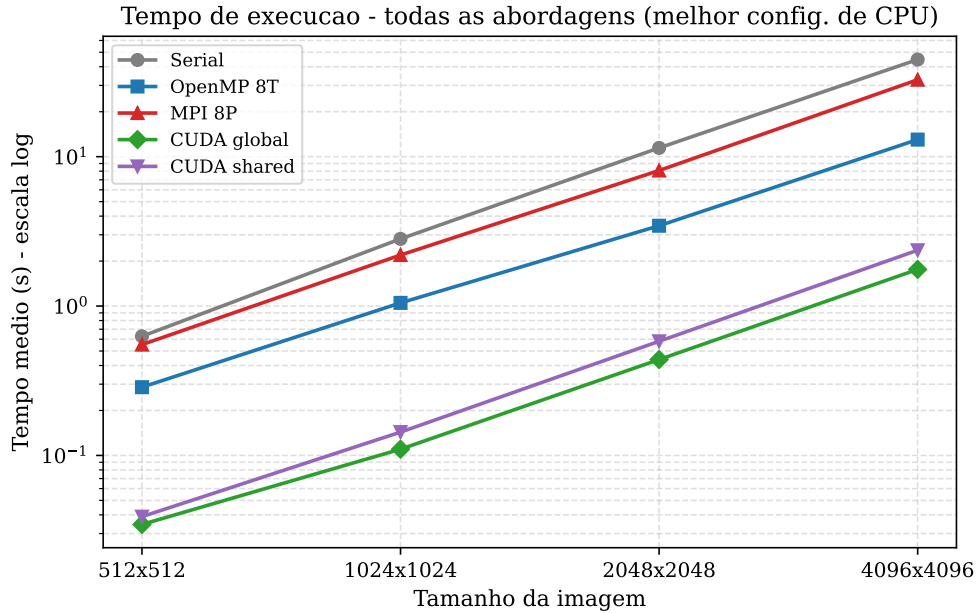


Figura 4: Tempo de execução das quatro abordagens por tamanho de imagem.

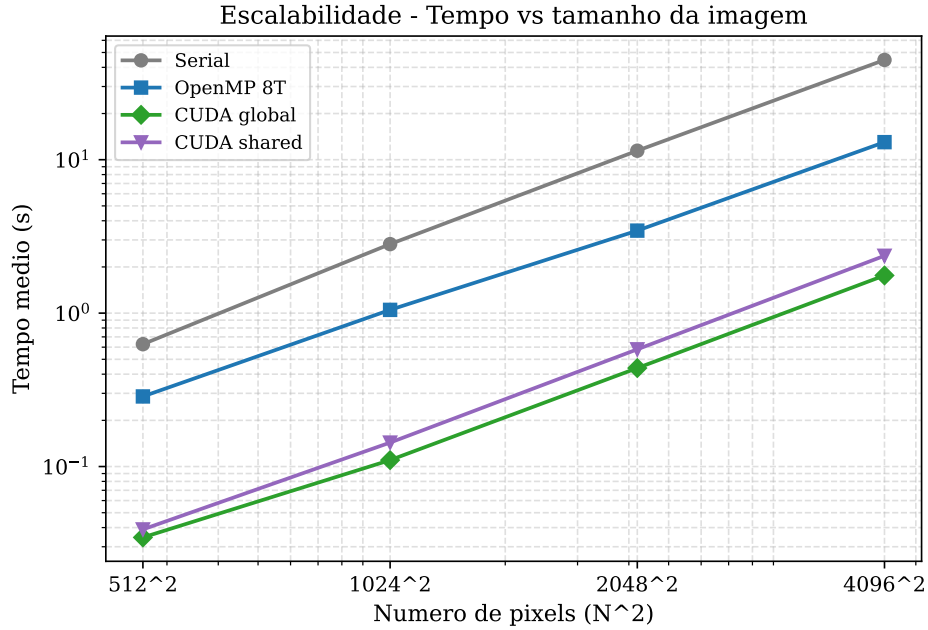


Figura 5: Escalabilidade: tempo em função do número de pixels (escala log-log).

O tempo cresce mais ou menos linear com o número de pixels tanto na CPU quanto na GPU (as retas ficam paralelas na escala *log-log* da Figura 5), o que é esperado, já que o trabalho por iteração é proporcional à área da imagem. A diferença é que as retas do CUDA ficam bem abaixo das de CPU, e o *speedup* se mantém alto na faixa toda.

O único ponto fora da curva é o  $512 \times 512$ , onde o *speedup* cai para  $18,2\times$  (contra cerca de  $26\times$  nos tamanhos médios). O motivo é o custo de lançar os *kernels*. A bateria dispara 1000 *kernels*, um por iteração, e cada lançamento tem um custo fixo de alguns microssegundos. No  $512 \times 512$  o laço todo leva cerca de 35 ms, uns  $35\mu s$  por iteração, então esse custo vira uma fatia relevante do tempo. No  $4096 \times 4096$  (1,76 s) ele some no meio. Não é falta de paralelismo, porque mesmo o  $512 \times 512$  gera  $32 \times 32 = 1024$  blocos, de sobra para ocupar os 16 SMs da placa. Ou seja, o que limita a escalabilidade é o custo fixo de lançamento nas imagens pequenas e a banda de memória nas grandes, não a capacidade de cálculo.

### 3.5 Análise de Ocupância e Configuração de Blocos (opcional/bônus)

**Ocupância teórica do bloco  $16 \times 16$ .** Na Turing (CC 7.5), cada SM aguenta até 1024 *threads* (32 *warps*) e 16 blocos. Com blocos de 256 *threads* (8 *warps*), o limite de *threads* deixa  $1024/256 = 4$  blocos por SM, ou seja  $4 \times 8 = 32$  *warps*, que é justamente o teto do SM. Isso dá ocupância teórica de 100%. A memória compartilhada que o `blur_shared` usa é pequena (cerca de 2,7 KB por bloco, contra 64 KB por SM), então não limita a ocupância. Esse ponto é central para entender o resultado: como o `blur_global` já chega na ocupância máxima e a GTX 1650 tem *cache* L1/textura que absorve o reuso do kernel  $3 \times 3$ , sobra pouca margem para o *tiling* manual da `shared` melhorar.

**Configuração de blocos ( $16 \times 16$  vs  $32 \times 32$ ).** Escolhemos o bloco  $16 \times 16$  (256 *threads*) por ser múltiplo de 32 e por deixar 4 blocos por SM, o que dá ao escalonador margem para intercalar blocos e esconder a latência de memória. Um bloco  $32 \times 32$  (1024 *threads*) também chegaria a 100% de ocupância em número de *threads*, mas com só 1 bloco por SM,

o que reduz essa margem, e ainda aumentaria o *tile* com halo na memória compartilhada. Por isso o  $16 \times 16$  costuma ser melhor para esse tipo de *stencil*. Uma varredura empírica de tamanhos de bloco com *profiler* fica como trabalho futuro.

*Observação:* não rodamos um *profiler* (Nsight Compute) neste ambiente WSL2. Os valores de ocupância acima são teóricos, calculados a partir dos limites da arquitetura Turing e da configuração de bloco usada.

**Investigação: Global vs Shared variando o kernel.** Um resultado chamou atenção. A versão **shared**, que a gente esperava ser a mais rápida por ser o análogo do *tiling*, perdeu para a **global** em todos os tamanhos de imagem com kernel  $3 \times 3$  (Tabela 2). Para entender, variamos o tamanho do kernel na imagem  $2048 \times 2048$  (1000 iterações, 3 repetições, fora da bateria oficial). O resultado está na Tabela 4 e na Figura 6.

Tabela 4: Global vs Shared variando o kernel ( $2048 \times 2048$ , 1000 iter, 3 rep).

| Kernel         | Global (s) | Shared (s) | Quem vence                       |
|----------------|------------|------------|----------------------------------|
| $3 \times 3$   | 0,452      | 0,585      | Global (+29%)                    |
| $5 \times 5$   | 0,852      | 0,917      | Global (+7,6%)                   |
| $7 \times 7$   | 1,249      | 1,218      | Shared (+2,5%)                   |
| $9 \times 9$   | 2,075      | 2,096      | Empate técnico ( $\approx 1\%$ ) |
| $11 \times 11$ | 2,800      | 2,758      | Shared (+1,5%)                   |

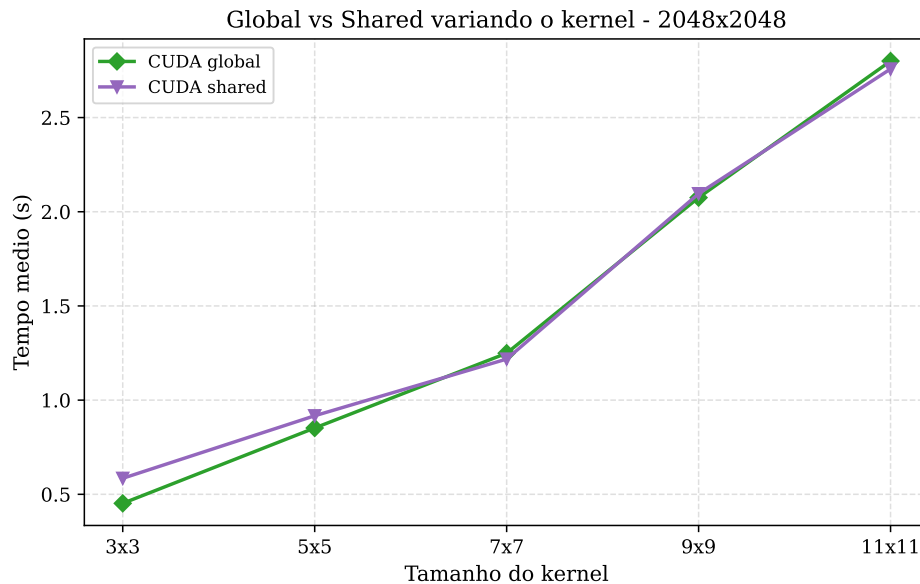


Figura 6: Tempo Global vs Shared em função do tamanho do kernel ( $2048 \times 2048$ ).

A tendência confirma a ideia: a vantagem da **shared** cresce com o tamanho do kernel. Até  $5 \times 5$  a **global** vence com folga (de 7,6% a 29%). A partir de mais ou menos  $7 \times 7$  vira empate técnico e a **shared** para de perder. Com só 3 repetições e diferenças de 1% a 2,5%, os pontos de  $7 \times 7$  em diante ficam dentro do ruído (por isso o  $9 \times 9$  aparece de leve a favor da **global**), então o mais honesto é dizer que a partir de  $7 \times 7$  as duas empatam, e não que existe um cruzamento exato. A causa está na Seção 4.4.

## 4 Discussão

### 4.1 Corretude

A diferença pequena das versões CUDA (4 pixels, 1 nível de cinza) é esperada e não é erro. A GPU usa *fused multiply-add* e soma os termos da convolução em uma ordem diferente da CPU, e em ponto flutuante a ordem das operações muda o último *bit*. Como no fim o valor volta para inteiro de 8 bits (0 a 255), essa diferença mínima às vezes arredonda para o vizinho. As duas versões CUDA deram o mesmo resultado entre si, o que reforça que o carregamento do *tile* com halo no `blur_shared` está certo.

### 4.2 Serial vs CUDA

O ganho é grande, de cerca de  $18\times$  a  $26\times$ . Ele vai muito além do que só o poder de cálculo explicaria, e a razão é a banda de memória. O filtro  $3 \times 3$  é *memory-bound* (poucas contas, muitos acessos), e a GPU tem memória GDDR com banda bem maior que a DDR4 do *host*, além de milhares de *threads* ao mesmo tempo que escondem a latência de acesso umas das outras. É esse gargalo que travava as versões de CPU, e é justo nele que a GPU se sai melhor.

### 4.3 CUDA vs OpenMP e MPI

Como as quatro versões rodaram no mesmo notebook, a comparação desta etapa é consistente. No  $4096 \times 4096$ , a CUDA global (1,76 s) é cerca de  $7,4\times$  mais rápida que o melhor OpenMP (13,0 s com 8 *threads*) e cerca de  $18,6\times$  mais rápida que o melhor MPI (32,7 s com 8 processos). A GPU ganha das duas abordagens de CPU em todos os tamanhos, e a diferença é grande porque o problema é limitado por memória, que é onde a GPU leva vantagem.

Vale notar que, neste hardware, o MPI ficou atrás do OpenMP, chegando a ser mais lento que o serial com 2 e 4 processos, o contrário do que vimos na Etapa 2 no Ryzen 7 5700G. Duas razões: este notebook tem só 4 núcleos físicos (metade do Ryzen), então o custo de comunicação pesa mais, e o transporte de memória compartilhada do OpenMPI costuma ser lento no WSL2, o que deixa a troca de halos de cada iteração ainda mais cara. A conclusão da Etapa 2 (MPI melhor que OpenMP) vale para aquele hardware, não para este.

### 4.4 Gargalos e por que Global venceu Shared

O gargalo principal continua sendo memória, não cálculo. Isso explica tanto os ganhos da GPU quanto o resultado meio contraintuitivo de a `global` vencer a `shared` com kernel  $3 \times 3$ :

- Com kernel  $3 \times 3$ , cada pixel é lido no máximo 9 vezes pelos vizinhos, ou seja, pouco reuso. A GTX 1650 tem *cache* L1/textura unificada que já absorve esse reuso sozinha, o que beneficia a `global` de graça.
- A `shared` paga um custo que a `global` não paga: um `__syncthreads()` por iteração (1000 barreiras de bloco) e o laço de carregar o *tile* com halo, que tem *branches* a mais nas bordas do bloco. Com kernel  $3 \times 3$  esse custo é maior que a banda que ela economiza.

- A partir de mais ou menos  $7 \times 7$  já tem reuso suficiente por pixel para a economia de tráfego compensar o custo da sincronização, e aí as duas se equilibram (Tabela 4).

Tem ainda um gargalo que não medimos aqui, a transferência *host* $\leftrightarrow$ *device*. Como o benchmark faz 1000 iterações para uma única cópia de ida e volta, esse custo se dilui. Em um cenário de uma passada só do filtro, porém, o tempo de cópia poderia dominar o tempo de *kernel*.

## 4.5 Impacto da configuração de blocos

A escolha do bloco  $16 \times 16$  está ligada direto ao bom desempenho da versão `global`. Ele chega a ocupância teórica de 100% na Turing e deixa 4 blocos por SM (a análise está na Seção 3.5). Em outras palavras, a placa já opera perto do seu limite de paralelismo sem nenhum esforço manual de *tiling*, o que é parte da explicação para a `shared` não compensar com kernel  $3 \times 3$ .

## 4.6 Melhorias futuras

- **Kernel maior** ( $7 \times 7$  ou mais): além de melhorar a razão cálculo/memória, é o regime onde o *tiling* em memória compartilhada passa a compensar.
- **Streams CUDA**: sobrepor transferência e computação, útil em cenários com poucas iterações, onde a cópia não se dilui.
- **Varredura de configuração de bloco** ( $8 \times 8$ ,  $32 \times 32$ ) com *profiler* (Nsight Compute) para medir ocupância e vazão de memória reais, em vez de estimativas teóricas.

## 5 Conclusão

Nesta etapa implementamos e avaliamos o benchmark de suavização Gaussiana em GPU com CUDA, em duas versões (memória global e compartilhada), e comparamos com o serial, o OpenMP e o MPI rodados de novo no mesmo notebook.

O principal aprendizado é que, para um problema *memory-bound* como esse, a GPU é muito mais rápida: de  $18\times$  a  $26\times$  sobre o serial, e cerca de  $7\times$  sobre a melhor versão de CPU (OpenMP com 8 *threads*). Esse ganho vem menos do poder de cálculo e mais da banda da memória GDDR e da capacidade de esconder latência com milhares de *threads*.

O resultado mais interessante foi a versão de memória compartilhada perder para a global com kernel  $3 \times 3$ . Ele mostra que otimizar memória em GPU não é automático: o *tiling* manual só compensa quando há reuso suficiente por pixel, o que, nesta GPU, só acontece a partir de kernels em torno de  $7 \times 7$ . Com o kernel pequeno da bateria, o *cache* L1 da própria placa já faz o serviço, e o custo de sincronização do *tiling* acaba atrapalhando mais do que ajudando.

Por fim, refazer todas as abordagens no mesmo hardware mostrou que conclusão de desempenho depende do ambiente: o MPI, que ganhava do OpenMP no Ryzen da Etapa 2, ficou atrás dele neste notebook de 4 núcleos no WSL2. Fica a lição de sempre reportar o *speedup* junto do hardware onde ele foi medido.

Entre os desafios que enfrentamos, os principais foram o conflito de *toolchain* entre o CUDA e o `glibc` novo do Ubuntu 26.04 (resolvido com o Toolkit 13.1 e a flag

-U\_GNU\_SOURCE) e a depuração do carregamento do *tile* com halo na memória compartilhada, que validamos pela igualdade de resultado entre as duas versões CUDA.

## 6 Referências

- Material de aula da disciplina DEC107, Processamento Paralelo.
- NVIDIA Corporation. *CUDA C++ Programming Guide*. Disponível em: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- NVIDIA Corporation. *CUDA C++ Best Practices Guide*. Disponível em: <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>
- NVIDIA Corporation. *Tuning CUDA Applications for Turing*. Disponível em: <https://docs.nvidia.com/cuda/tuning-tuning-guide/>
- NOTHINGS. *stb\_image* e *stb\_image\_write*, bibliotecas single-file de domínio público para C/C++. Disponível em: <https://github.com/nothings/stb>

# Declaração de Transparência e Uso de Inteligência Artificial

Eu, **Lucas Luige Costa Miranda**, regularmente matriculado(a) no curso de Ciência da Computação, declaro para os devidos fins que, na elaboração do trabalho/projeto intitulado “**Projeto 1 — Benchmark de Suavização/Desfoque Gaussiano Iterativo: Implementação paralela em GPU com CUDA. (Etapa 3)**”, submetido como requisito para a disciplina/atividade DEC107 — Processamento Paralelo, adotei as seguintes práticas em relação ao uso de ferramentas de Inteligência Artificial (IA) generativa:

**(X) Utilizei ferramentas de Inteligência Artificial como recurso de apoio.**

**Detalhamento do Uso:** As ferramentas utilizadas foram: **Claude (Anthropic)**. O uso se deu exclusivamente nas seguintes etapas do projeto:

- Auxílio na depuração (*debugging*) e estruturação dos *kernels* CUDA (mapeamento de *threads*, carregamento do *tile* com halo na memória compartilhada, uso da memória constante);
- Resolução do conflito de *toolchain* entre o CUDA e o `glibc` do Ubuntu 26.04;
- Discussão e interpretação dos resultados de desempenho (*speedup*, ocupância, e o comportamento global vs shared em função do kernel);
- Revisão e organização dos arquivos de análise comparativa (Serial vs. OpenMP vs. MPI vs. CUDA) e auxílio na estruturação do relatório em L<sup>A</sup>T<sub>E</sub>X.

**Termo de Responsabilidade:** Declaro estar ciente de que ferramentas de IA não são isentas de erros, vieses ou alucinações. Sendo assim, atesto que toda a responsabilidade pela autoria, integridade, originalidade e exatidão do conteúdo submetido é exclusivamente minha. Confirmo que todo o código-fonte, dados de testes e textos gerados ou modificados com o auxílio de IA foram rigorosamente revisados, testados e validados por mim, garantindo que os resultados refletem meu próprio entendimento e esforço intelectual, sem incorrer em plágio intelectual ou acadêmico.

Itabuna/BA, 04 de julho de 2026

Lucas Luige Costa Miranda  
Matrícula: 202220035

Eu, **Henrique Cerqueira Fadigas**, regularmente matriculado(a) no curso de Ciência da Computação, declaro para os devidos fins que, na elaboração do trabalho/projeto intitulado “**Projeto 1 — Benchmark de Suavização/Desfoque Gaussiano Iterativo: Implementação paralela em GPU com CUDA. (Etapa 3)**”, submetido como requisito para a disciplina/atividade DEC107 — Processamento Paralelo, adotei as seguintes práticas em relação ao uso de ferramentas de Inteligência Artificial (IA) generativa:

**(X) Utilizei ferramentas de Inteligência Artificial como recurso de apoio.**

**Detalhamento do Uso:** As ferramentas utilizadas foram: **Claude (Anthropic)**. O uso se deu exclusivamente nas seguintes etapas do projeto:

- Auxílio na depuração (*debugging*) e estruturação dos *kernels* CUDA (memória global, compartilhada e constante, configuração de blocos e grade);
- Resolução do conflito de *toolchain* entre o CUDA e o `glibc` do Ubuntu 26.04;
- Discussão e interpretação dos resultados de desempenho (*speedup*, ocupância, e o comportamento global vs shared em função do kernel);
- Revisão e organização dos arquivos de análise comparativa (Serial vs. OpenMP vs. MPI vs. CUDA) e auxílio na estruturação do relatório em  $\text{\LaTeX}$ .

**Termo de Responsabilidade:** Declaro estar ciente de que ferramentas de IA não são isentas de erros, vieses ou alucinações. Sendo assim, atesto que toda a responsabilidade pela autoria, integridade, originalidade e exatidão do conteúdo submetido é exclusivamente minha. Confirmo que todo o código-fonte, dados de testes e textos gerados ou modificados com o auxílio de IA foram rigorosamente revisados, testados e validados por mim, garantindo que os resultados refletem meu próprio entendimento e esforço intelectual, sem incorrer em plágio intelectual ou acadêmico.

Itabuna/BA, 04 de julho de 2026

Henrique Cerqueira Fadigas  
Matrícula: 202220028